D09
DBMS

Experiment No: 2
Implementation of DDL commands of SQL with suitable examples.

(Scenario: Online Banking System)

1. **CREATE TABLE**

Query:

```
create table Customers(customer_id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(50));
```

Output:



2. **ALTER TABLE – Add a new column**

Query:

```
ALTER TABLE Customers ADD phone VARCHAR(15);
```

Output:

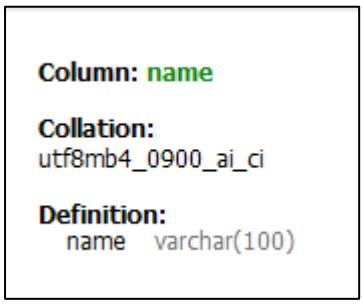


3. **ALTER TABLE – Modify column**

Query:

```
ALTER TABLE Customers MODIFY name VARCHAR(100);
```

Output:

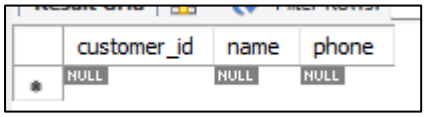


4. **ALTER TABLE – Drop column**

Ouery:

```
ALTER TABLE Customers DROP COLUMN email;
```

Output:

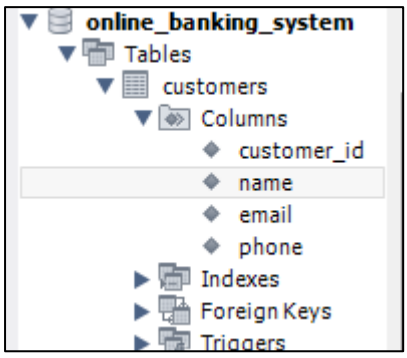


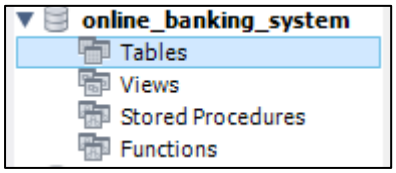
5. **DROP TABLE**

Ouery:

```
DROP TABLE Customers;
```

Output:

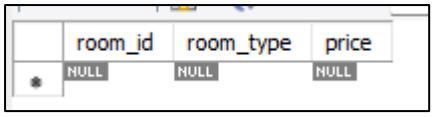




(Scenario: Hotel Management System)

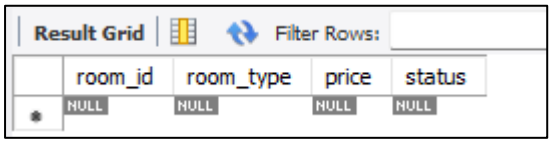
1. CREATE TABLE

```
CREATE TABLE Rooms (
   room_id INT PRIMARY KEY,
   room_type VARCHAR(50),
   price DECIMAL(10,2)
);
```



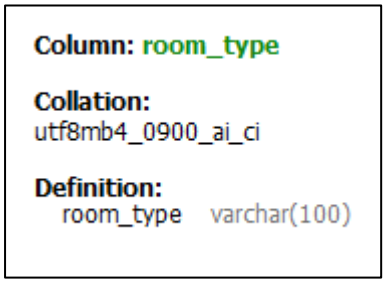
2. ALTER TABLE – Add a new column

```
ALTER TABLE Rooms
ADD status VARCHAR(20);
```



3. ALTER TABLE – Modify column

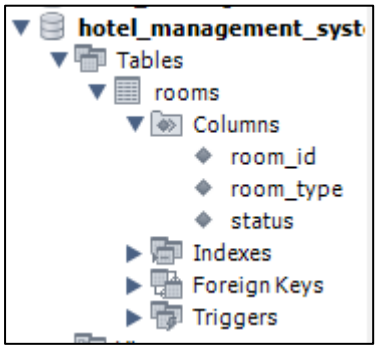
```
ALTER TABLE Rooms
MODIFY room_type VARCHAR(100);
```

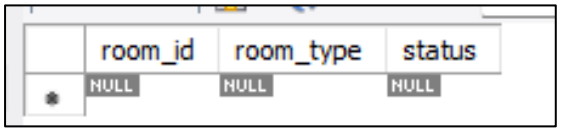


4. ALTER TABLE – Drop column

```
ALTER TABLE Rooms

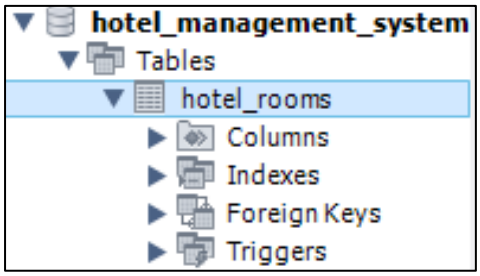
DROP COLUMN price;
```





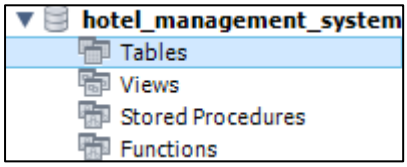
5. RENAME TABLE:

```
ALTER TABLE Rooms RENAME TO Hotel_Rooms;
```



6. DROP TABLE

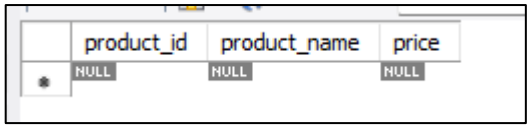
```
DROP TABLE hotel_rooms;
```



Scenario: E-Commerce System

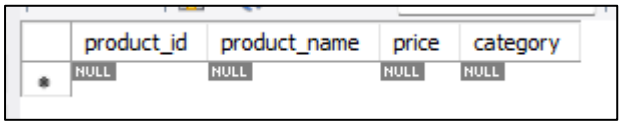
1. CREATE TABLE

```
CREATE TABLE Products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(50),
   price DECIMAL(10,2)
);
```



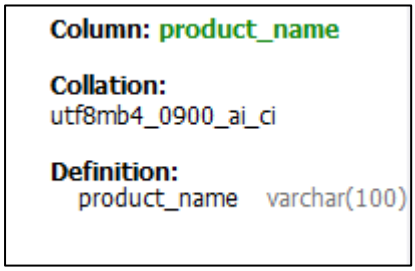
2. ALTER TABLE – Add a new column

```
ALTER TABLE Products
ADD category VARCHAR(50);
```



3. ALTER TABLE – Modify a column

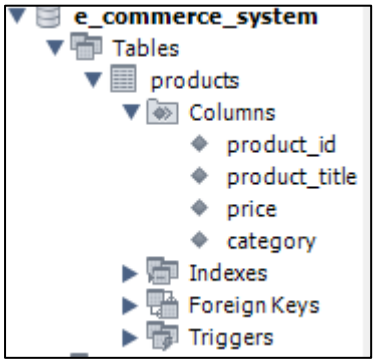
```
ALTER TABLE Products
MODIFY product_name VARCHAR(100);
```



4. ALTER TABLE – Rename a column

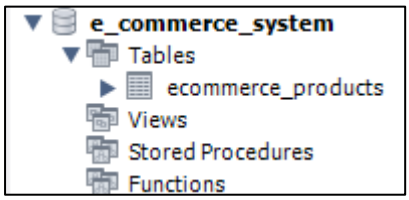
```
ALTER TABLE Products

RENAME COLUMN product_name TO product_title;
```



5. RENAME TABLE

```
RENAME TABLE Products TO Ecommerce_Products;
```



TCOD09

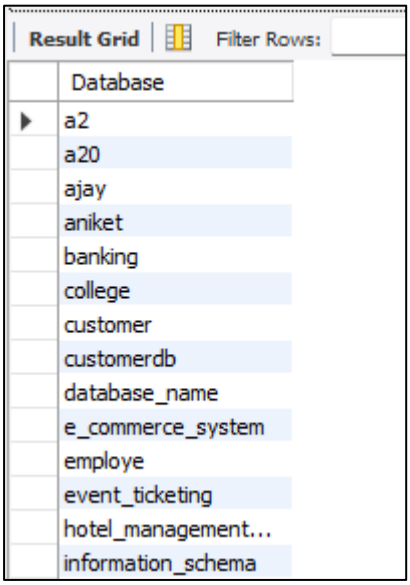
Experiment No : 3

: To perform the following SQL operations:

1. **Viewing all databases.**

```
show databases;
```

Output:

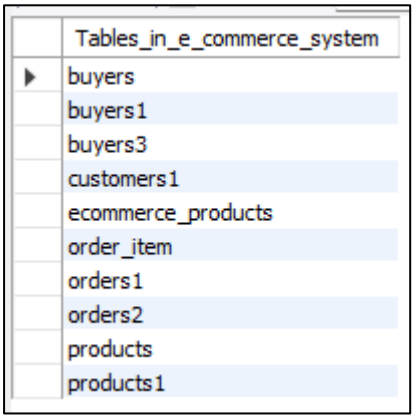


Database
a2
a20
ajay
aniket
banking
college
customer
customerdb
database_name
e_commerce_system
employe
event_ticketing
hotel_management...
information_schema

2. **Viewing all tables in a database.**

```
use e_commerce_system;
show tables;
```

Output:



Tables_in_e_commerce_system
buyers
buyers1
buyers3
customers1
ecommerce_products
order_item
orders1
orders2
products
products1

TCOD09

3. **Updating records in a table.**

```
update buyers
SET Name = Updated_Name
WHERE Customer_id=01;
```

Output:

Before:

Customer_id	Name	email
1	Demo1	abc@gmail.com
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
NULL	NULL	NULL

After:

Customer_id	Name	email
1	Updated_Name	abc@gmail.com
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
NULL	NULL	NULL

4. **Deleting records from a table.**

```
delete from buyers
WHERE Customer_id=01;
```

Output:

Before:

Customer_id	Name	email
1	Updated_Name	abc@gmail.com
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
NULL	NULL	NULL

After:

Customer_id	Name	email
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
NULL	NULL	NULL

TCOD09

Experiment No:4

Implementation of DML commands of SQL with suitable examples

1. **Insert**

```
insert into buyers (Customer_id,Name,email)
values (121,'Inserted_User','inserteduser91@gmail.com');

select * from buyers:
```

Output:

Before:

Customer_id	Name	email
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
NULL	NULL	NULL

After:

Customer_id	Name	email
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
121	Inserted_User	inserte@gmail.com
NULL	NULL	NULL

2. **Update**

```
update buyers
SET Name = 'Updated_User'
WHERE Customer_id = 2;

Select * from buyers;
```

Output:

Before:

Customer_id	Name	email
2	User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
121	Inserted_User	inserte@gmail.com
NULL	NULL	NULL

After:

Customer_id	Name	email
2	Updated_User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
121	Inserted_User	inserte@gmail.com
NULL	NULL	NULL

TCOD09

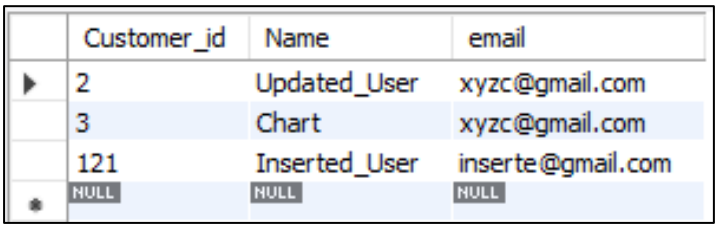
3. **Delete**.

```
delete from buyers
WHERE Customer_id = 2;

Select * from buyers;
```

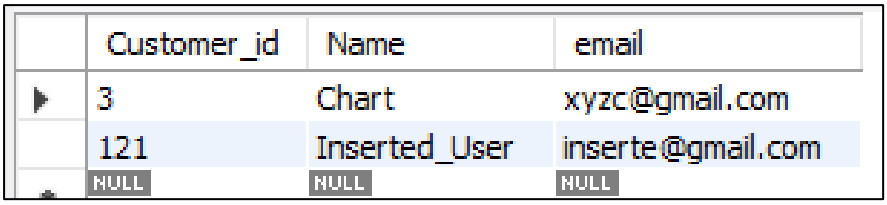
Output:

Before:



Customer_id	Name	email
2	Updated_User	xyzc@gmail.com
3	Chart	xyzc@gmail.com
121	Inserted_User	inserte@gmail.com
NULL	NULL	NULL

After:



Customer_id	Name	email
3	Chart	xyzc@gmail.com
121	Inserted_User	inserte@gmail.com
NULL	NULL	NULL

TCOD09

Experiment No : 5

For a given set of relation schemes, create tables and perform the following Simple Queries:

EmpID	Name	Department	Salary	JoiningDate
101	Rahul	HR	35000.00	2022-01-15
102	Priya	IT	50000.00	2021-07-20
103	Amit	IT	60000.00	2020-05-10
104	Neha	Sales	45000.00	2023-03-18
105	Rohit	HR	40000.00	2022-09-12
NULL	NULL	NULL	NULL	NULL

1. **Aggregate Functions**

```
SELECT COUNT(*) AS TotalEmployees FROM Employee;
```

Output:

TotalEmployees
5

```
SELECT SUM(Salary) AS TotalSalary FROM Employee;
```

Output:

TotalSalary
230000.00

```
SELECT AVG(Salary) AS AverageSalary FROM Employee;
```

Output:

AverageSalary
46000.000000

```
SELECT MAX(Salary) AS HighestSalary FROM Employee;
```

Output:

HighestSalary
60000.00

```
SELECT MIN(Salary) AS LowestSalary FROM Employee;
```

Output:

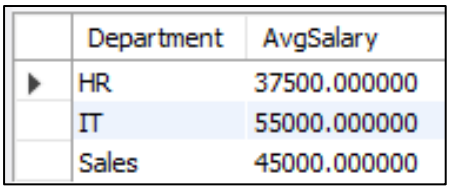
LowestSalary
35000.00

TCOD09

2. GROUP BY Clause

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department;
```

Output:

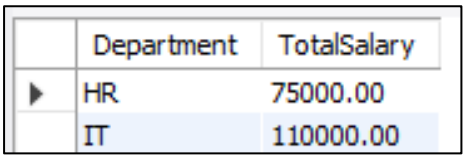


	Department	AvgSalary
▶	HR	37500.000000
	IT	55000.000000
	Sales	45000.000000

3. HAVING Clause

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Department
HAVING SUM(Salary) > 70000;
```

Output:

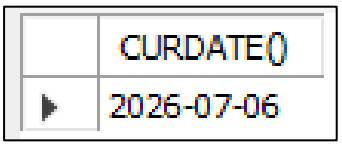


	Department	TotalSalary
▶	HR	75000.00
	IT	110000.00

4. Date Functions

```
SELECT CURDATE();
```

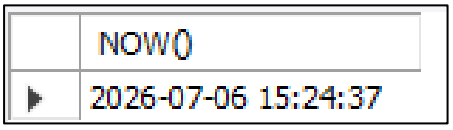
Output:



	CURDATE()
▶	2026-07-06

```
SELECT NOW();
```

Output:



	NOW()
▶	2026-07-06 15:24:37

TCOD09

```
SELECT YEAR(JoiningDate) FROM Employee;
```

Output:

	YEAR(JoiningDate)
▶	2022
	2021
	2020
	2023
	2022

```
SELECT MONTH(JoiningDate) FROM Employee;
```

Output:

	MONTH(JoiningDate)
▶	1
	7
	5
	3
	9

```
SELECT DAY(JoiningDate) FROM Employee;
```

Output:

	MONTH(JoiningDate)
▶	1
	7
	5
	3
	9

TCOD09

5. String Functions

```
SELECT UPPER(Name) FROM Employee;
```

Output:

	UPPER(Name)
▶	RAHUL
	PRIYA
	AMIT
	NEHA
	ROHIT

```
SELECT LOWER(Name) FROM Employee;
```

Output:

	LOWER(Name)
▶	rahul
	priya
	amit
	neha
	rohit

```
SELECT LENGTH(Name) FROM Employee;
```

Output:

	LENGTH(Name)
▶	5
	5
	4
	4
	5

```
SELECT CONCAT(Name,' - ',Department) FROM Employee;
```

Output:

	CONCAT(Name,' - ',Department)
▶	Rahul - HR
	Priya - IT
	Amit - IT
	Neha - Sales
	Rohit - HR

```
SELECT SUBSTRING(Name,1,3) FROM Employee;
```

Output:

	SUBSTRING(Name,1,3)
▶	Rah
	Pri
	Ami
	Neh
	Roh

TCOD09

6. Math Functions

```
SELECT ROUND(123.456,2);
```

Output:

	ROUND(123.456,2)
▶	123.46

```
SELECT CEIL(45.12);
```

Output:

	CEIL(45.12)
▶	46

```
SELECT FLOOR(45.98);
```

Output:

	FLOOR(45.98)
▶	45

```
SELECT ABS(-200);
```

Output:

	ABS(-200)
▶	200

```
SELECT SQRT(144);
```

Output:

	SQRT(144)
▶	12

Experiment No: 6
Implementation of Different Types of Joins
Employee:

Emp_ID	Emp_Name	Department_ID	Salary
101	Rahul	1	50000.00
102	Rishab	2	40000.00
103	Sima	3	15000.00
104	kishor	4	55000.00
105	ishan	5	75000.00
106	Rakesh	6	50000.00
NULL	NULL	NULL	NULL

Department:

Department_ID	Department_Name
1	HR
2	Finance
3	IT
4	Marketing
5	Manager
NULL	NULL

1. INNER JOIN

```
SELECT E.Emp_ID,
    E.Emp_Name,
    D.Department_Name,
    E.Salary
FROM Employee E
INNER JOIN Department D
ON E.Department_ID = D.Department_ID;
```

Output:

Emp_ID	Emp_Name	Department_Name	Salary
101	Rahul	HR	50000.00
102	Rishab	Finance	40000.00
103	Sima	IT	15000.00
104	kishor	Marketing	55000.00
105	ishan	Manager	75000.00

2. LEFT OUTER JOIN

```
SELECT E.Emp_ID,
    E.Emp_Name,
    D.Department_Name
FROM Employee E
LEFT JOIN Department D
ON E.Department_ID = D.Department_ID;
```

Output:

Emp_ID	Emp_Name	Department_Name
101	Rahul	HR
102	Rishab	Finance
103	Sima	IT
104	kishor	Marketing
105	ishan	Manager
106	Rakesh	NULL

3. RIGHT OUTER JOIN

```
SELECT E.Emp_ID,
    E.Emp_Name,
    D.Department_Name
FROM Employee E
RIGHT JOIN Department D
ON E.Department_ID = D.Department_ID;
```

Output:

Emp_ID	Emp_Name	Department_Name
101	Rahul	HR
105	ishan	Finance
102	Rishab	Finance
103	Sima	IT
104	kishor	Marketing
NULL	NULL	Manager

4. NATURAL JOIN

```
SELECT * FROM Employee NATURAL JOIN Department;
```

Output:

Department_ID	Emp_ID	Emp_Name	Salary	Department_Name
1	101	Rahul	50000.00	HR
2	102	Rishab	40000.00	Finance
3	103	Sima	15000.00	IT
4	104	kishor	55000.00	Marketing
2	105	ishan	75000.00	Finance

4. CROSS JOIN

```
Select E.Emp_ID,
E.Emp_Name,
D.Department_ID,
D.Department_Name
From Employee E
Cross Join Department D;
```

Output:

Emp_ID	Emp_Name	Department_ID	Department_Name
101	Rahul	5	Manager
101	Rahul	4	Marketing
101	Rahul	3	IT
101	Rahul	2	Finance
101	Rahul	1	HR
102	Rishab	5	Manager
102	Rishab	4	Marketing
102	Rishab	3	IT
102	Rishab	2	Finance
102	Rishab	1	HR
103	Sima	5	Manager
103	Sima	4	Marketing
103	Sima	3	IT
103	Sima	2	Finance
103	Sima	1	HR
104	kishor	5	Manager
104	kishor	4	Marketing
104	kishor	3	IT
104	kishor	2	Finance
104	kishor	1	HR
105	ishan	5	Manager
105	ishan	4	Marketing
105	ishan	3	IT
105	ishan	2	Finance

5. SELF JOIN:

Emp_ID	Emp_Name	Department_ID	Salary	Manager_ID
101	Rahul	1	50000.00	NULL
102	Rishab	2	40000.00	101
103	Sima	3	15000.00	101
104	kishor	4	55000.00	102
105	ishan	2	75000.00	102
106	Rakesh	6	50000.00	NULL
NULL	NULL	NULL	NULL	NULL

Query:

```
SELECT E1.Emp_Name AS Employee,
      E2.Emp_Name AS Manager
FROM Employee E1
LEFT JOIN Employee E2
ON E1.Manager_ID = E2.Emp_ID;
```

Output:

Employee	Manager
Rahul	NULL
Rishab	Rahul
Sima	Rahul
kishor	Rishab
ishan	Rishab
Rakesh	NULL

Experiment No: 7

To perform SQL query that demonstrate following: Search conditions, Summary queries, Sub- queries, Subqueries- With IN clause, With EXISTS clause

1. **Search Conditions**

- **Students scoring more than 80**

```
SELECT * FROM Student WHERE Marks > 80;
```

Output:

StudentID	StudentName	Age	Gender	DeptID	Marks
1	Amit	20	Male	101	85
2	Priya	19	Female	101	92
4	Sneha	20	Female	103	88
6	Neha	21	Female	104	95
8	Pooja	19	Female	103	81
NULL	NULL	NULL	NULL	NULL	NULL

- **Students from Computer Department**

```
SELECT * FROM Student WHERE DeptID = 101;
```

Output:

StudentID	StudentName	Age	Gender	DeptID	Marks
1	Amit	20	Male	101	85
2	Priya	19	Female	101	92
7	Arjun	20	Male	101	70
NULL	NULL	NULL	NULL	NULL	NULL

- **Students between 70 and 90 marks**

```
SELECT * FROM Student WHERE Marks BETWEEN 70 AND 90;
```

Output:

StudentID	StudentName	Age	Gender	DeptID	Marks
1	Amit	20	Male	101	85
3	Rahul	21	Male	102	75
4	Sneha	20	Female	103	88
7	Arjun	20	Male	101	70
8	Pooja	19	Female	103	81
NULL	NULL	NULL	NULL	NULL	NULL

2. Summary Queries

- **Total Students**

```
SELECT COUNT(*) AS TotalStudents FROM Student;
```

Output:

	TotalStudents
▶	8

- **Average Marks**

```
SELECT AVG(Marks) AS AverageMarks FROM Student;
```

Output:

	AverageMarks
▶	81.3750

- **Highest Marks**

```
SELECT MAX(Marks) AS HighestMarks FROM Student;
```

Output:

	HighestMarks
▶	95

- **Department-wise Average Marks**

```
SELECT DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY DeptID;
```

Output:

	DeptID	AvgMarks
▶	101	82.3333
	102	70.0000
	103	84.5000
	104	95.0000

3. Subquery

- **Students scoring above average**

```
SELECT StudentName, Marks
FROM Student
WHERE Marks >
(
  SELECT AVG(Marks)
  FROM Student
);
```

Output:

	StudentName	Marks
▶	Amit	85
	Priya	92
	Sneha	88
	Neha	95

4. **Subquery with IN Clause**

- **Students belonging to departments located in Building A or Building C**

```
SELECT StudentName, DeptID
FROM Student
WHERE DeptID IN
(
  SELECT DeptID
  FROM Department
  WHERE Location IN ('Building A', 'Building C')
);
```

Output:

	StudentName	DeptID
▶	Amit	101
	Priya	101
	Sneha	103
	Arjun	101
	Pooja	103

5. **Subquery with EXISTS Clause**

- **Display departments that have at least one student**

```
SELECT DeptName
FROM Department D
WHERE EXISTS
(
  SELECT *
  FROM Student S
  WHERE S.DeptID = D.DeptID
);
```

Output:

	DeptName
▶	Computer
	Mechanical
	Electrical
	Civil

Experiment No: 8

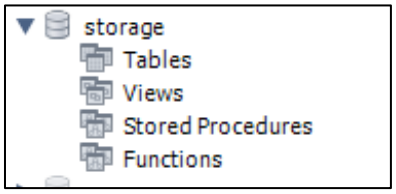
To perform the following database administration tasks:

- **Creating Database / Table Space**
- **Managing Users (Create User, Delete User)**
- **Managing Roles (Grant, Revoke)**

1. Creating Database

```
CREATE DATABASE STORAGE;
```

Output:



2. Creating Table Space

```
CREATE TABLE COLLEGE (
  COLLEGE_ID INT PRIMARY KEY,
  COLLEGE_NAME VARCHAR(100) NOT NULL,
  LOCATION VARCHAR(100),
  ESTABLISHED_YEAR INT,
  PHONE VARCHAR(15)
);

INSERT INTO college
VALUES (101, 'ABC College of Engineering', 'Mumbai', 1995, '9876543210');

INSERT INTO college
VALUES (102, 'XYZ College', 'Pune', 2001, '9876501234');
```

Output:

college_id	college_name	location	established_year	phone
101	ABC College of Engineering	Mumbai	1995	9876543210
102	XYZ College	Pune	2001	9876501234
NULL	NULL	NULL	NULL	NULL

3. **Creating Create User**

```
CREATE USER USER1 IDENTIFIED BY '1234';
```

Output:

```
2  09:01:08  create user user1 identified by '1234'
```

4. **Deleting User**

```
DROP USER USER1;
```

Output:

```
20  09:32:22  drop user user1
```

5. **Grant Command**

```
GRANT INSERT ON COLLEGE TO USER1;
```

Output:

```
22  09:35:51  grant insert on college to user1
```

6. **Revoke Command**

```
REVOKE INSERT ON COLLEGE FROM USER1;
```

Output:

```
23  09:41:56  revoke insert on college from user1
```

From User1@loaclhost:

```
INSERT INTO COLLEGE
VALUES (107, 'JSPM COLLEGE OF ENGINEERING', 'WAGHOLI,PUNE', 2005, '98766534410');
```

Output:

college_id	college_name	location	established_year	phone
101	ABC College of Engineering	Mumbai	1995	9876543210
102	XYZ College	Pune	2001	9876501234
103	College	wagholi	2003	98765012454
104	GHRCMP	Wagholi,pune	2001	9123456789
105	moze College of Engineering	pune	2001	98765434410
106	JSPM College of Engineering	wagholi,pune	2005	98766534410
107	JSPM College of Engineering	wagholi,pune	2005	98766534410

```
SELECT * FROM COLLEGE;
```

```
10  10:32:10  select * from college LIMIT 0, 1000
```

Experiment No: 9
Implementation of Database Backup & Recovery commands, Rollback, Commit, Savepoint.

1. **Creating Database**

```
CREATE DATABASE college;
```

Output:

	Database
▶	bank
	college
	ecommerce_db
	information_schema
	mydb

2. **Creating Table**

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  marks INT
);
```

Output:

	Field	Type	Null	Key	Default	Extra
▶	id	int	NO	PRI	NULL	
	name	varchar(50)	YES		NULL	
	marks	int	YES		NULL	

3. **Inserting into Table**

```
INSERT INTO students VALUES
(1, 'Rahul', 85),
(2, 'Priya', 90),
(3, 'Amit', 75);
```

Output:

	id	name	marks
▶	1	Rahul	85
	2	Priya	90
	3	Amit	75
*	NULL	NULL	NULL

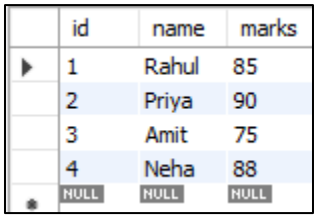
4. **COMMIT**

```
CSTART TRANSACTION;

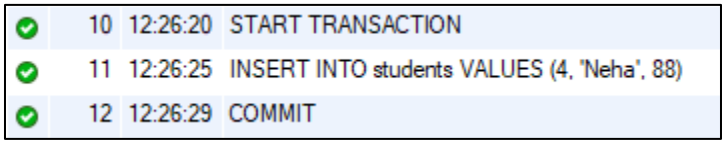
INSERT INTO students VALUES (4, 'Neha', 88);

COMMIT;;
```

Output:



id	name	marks
1	Rahul	85
2	Priya	90
3	Amit	75
4	Neha	88
NULL	NULL	NULL



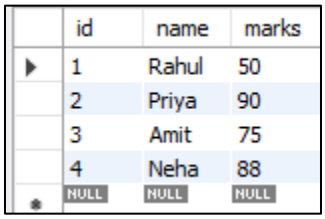
10	12:26:20	START TRANSACTION
11	12:26:25	INSERT INTO students VALUES (4, 'Neha', 88)
12	12:26:29	COMMIT

5. **ROLLBACK**

```
 START TRANSACTION;
UPDATE students
SET marks = 50
WHERE id = 1;
ROLLBACK;
```

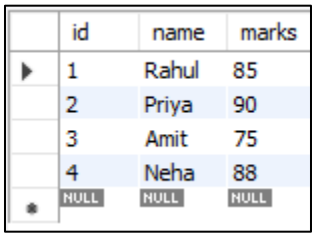
Output:

Before RollBack:



id	name	marks
1	Rahul	50
2	Priya	90
3	Amit	75
4	Neha	88
NULL	NULL	NULL

After Roll Back:



id	name	marks
1	Rahul	85
2	Priya	90
3	Amit	75
4	Neha	88
NULL	NULL	NULL

6. SAVEPOINT

```
START TRANSACTION;

UPDATE students
SET marks = 95
WHERE id = 1;

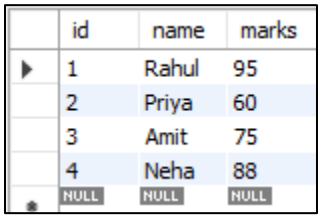
SAVEPOINT sp1;

UPDATE students
SET marks = 60
WHERE id = 2;

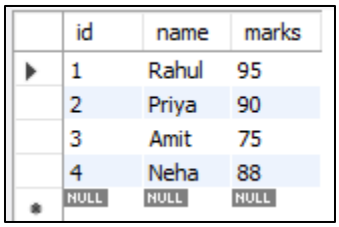
ROLLBACK TO SAVEPOINT sp1;
COMMIT;
```

Output:

Savepoint:

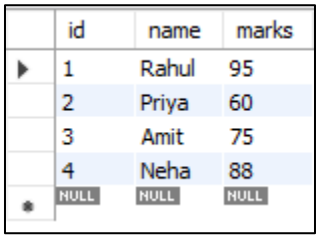


id	name	marks
1	Rahul	95
2	Priya	60
3	Amit	75
4	Neha	88
NULL	NULL	NULL



id	name	marks
1	Rahul	95
2	Priya	90
3	Amit	75
4	Neha	88
NULL	NULL	NULL

Rollback:



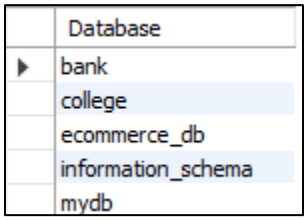
id	name	marks
1	Rahul	95
2	Priya	60
3	Amit	75
4	Neha	88
NULL	NULL	NULL

Experiment No: 10
Create View and index for database tables with a large number of records.

1. **Creating Database**

```
CREATE DATABASE college;
```

Output:

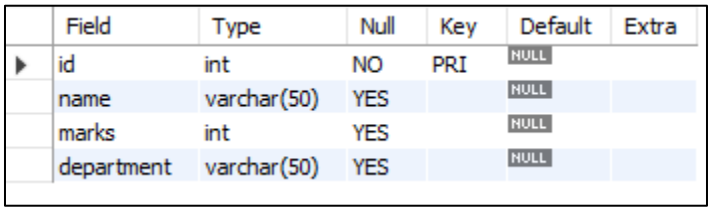


Database
bank
college
ecommerce_db
information_schema
mydb

2. **Creating Table**

```
CREATE TABLE student (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  marks INT,
  department VARCHAR(50)
);
```

Output:

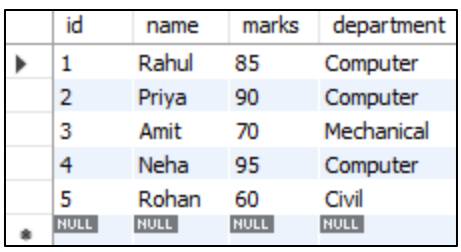


Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
marks	int	YES		NULL	
department	varchar(50)	YES		NULL	

3. **Inserting into Table**

```
insert INTO student VALUES
(1, 'Rahul', 85, 'Computer'),
(2, 'Priya', 90, 'Computer'),
(3, 'Amit', 70, 'Mechanical'),
(4, 'Neha', 95, 'Computer'),
(5, 'Rohan', 60, 'Civil');
```

Output:



id	name	marks	department
1	Rahul	85	Computer
2	Priya	90	Computer
3	Amit	70	Mechanical
4	Neha	95	Computer
5	Rohan	60	Civil
NULL	NULL	NULL	NULL

4. **Create a View**

```
CREATE VIEW student_view AS
SELECT id, name, marks
FROM students
WHERE marks >= 80;
```

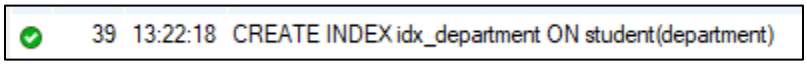
Output:

	id	name	marks
▶	1	Rahul	95
	4	Neha	88

5. **Create an Index**

```
CREATE INDEX idx_department
ON students(department);
```

Output:



Check the Index:

```
SHOW INDEX FROM student;
```

	Table	Non_unique	Key_name	Seq_in_index	Column_name
▶	student	0	PRIMARY	1	id
	student	1	idx_department	1	department

Check Whether Index Is Used :

```
EXPLAIN
SELECT * FROM students
WHERE department = 'Computer';
```

Output:

	id	select_type	table	partitions	type	possible_keys
▶	1	SIMPLE	student	NULL	ref	idx_department